Vulnerability Notes

Path Traversal

Concepts, Attack Techniques, Detection, and Defenses

Reference
PortSwigger Web Security Academy

Prepared by
Youssef Reda

Contents

Path Traversal (Directory Traversal) **1**
- Definition . . . 1
- How It Works . . . 1
- Why It Is Called Directory Traversal . . . 2
- Potential Impact . . . 2
  - Application Code and Data . . . 2
  - Back-End Credentials and Secrets . . . 2
  - Operating System Files . . . 3
  - Arbitrary File Writing . . . 3

Reading Arbitrary Files via Path Traversal **4**
- Overview . . . 4
- Example Scenario . . . 4
- How the Application Reads the File . . . 4
- Exploiting the Vulnerability . . . 4
- Understanding Path Traversal Sequences . . . 5
- Reading Arbitrary Files . . . 6
- Example Target: `/etc/passwd` . . . 6
- Windows Path Traversal . . . 6

Path Traversal: Common Obstacles and Bypass Techniques **7**
- Overview . . . 7
- Request Processing and Filtering . . . 7
- Absolute Path Bypass . . . 7
  - Problem . . . 7
  - Bypass . . . 7
  - Why It Works . . . 7
- Nested Traversal Sequences . . . 8
  - Problem . . . 8
  - Bypass . . . 8
  - Why It Works . . . 8
- URL Encoding and Double URL Encoding . . . 9
  - URL Encoding . . . 9
  - Double URL Encoding . . . 9
  - Non-standard Encodings . . . 9
  - Burp Suite . . . 9
- Validation of the Start of a Path . . . 10
  - Problem . . . 10
  - Bypass . . . 10
  - Why It Works . . . 10

File Extension Validation and Null Byte Injection 10
Problem 10
Historical Null Byte Bypass 11
Modern Considerations 11
Summary of Bypass Techniques 11

Path Traversal Prevention **12**
Avoid User-Controlled File Paths 12
Validate Input and Use a Trusted Base Directory 12
Canonicalize and Verify Directory Containment 12
Java Example 13
Best Practices 13

Path Traversal (Directory Traversal)

Definition

Path Traversal, also known as **Directory Traversal**, is a web vulnerability that allows an attacker to access files and directories outside the application's intended directory by manipulating user-controlled file paths. Depending on the application's functionality and permissions, this vulnerability may allow attackers to read sensitive files or, in some cases, write files to unintended locations.

How It Works

Applications sometimes use user-supplied input to construct filesystem paths without properly validating the input.

For example:

```
$file = $_GET['file'];
include("images/" . $file);
```

A normal request supplies a filename such as:

```
?file=cat.jpg
```

The application accesses:

```
/images/cat.jpg
```

If the application does not properly validate the supplied filename, an attacker may provide directory traversal sequences such as:

```
../../../etc/passwd
```

The resulting path becomes:

```
/images/../../../etc/passwd
```

The filesystem resolves each `../` sequence by moving to the parent directory. The resulting path may resolve to:

```
/etc/passwd
```

Consequently, the attacker may access files outside the intended directory.

Why It Is Called Directory Traversal

The attack uses path traversal sequences, most commonly `../` on Unix-like systems, to navigate from the intended directory to its parent directories until reaching a target file.

For example:

```
images/cat.jpg
       |
       | ../
       v
Parent directory
       |
       | ../
       v
Another parent directory
       |
       v
Target file
```

On Windows, traversal may also use backslashes (`..\`), depending on how the application and filesystem handle paths.

Potential Impact

The impact depends on the files accessible to the application process and the operations exposed by the vulnerable functionality.

Application Code and Data

An attacker may access files belonging to the application, including:

- Source code
- Configuration files
- Log files and backups
- Database files
- Uploaded files

For example, exposing `config.php` may reveal sensitive application settings or credentials.

Back-End Credentials and Secrets

Files such as `.env` may contain credentials used by the application to communicate with other systems, including:

- Database usernames and passwords
- API keys and access tokens
- SSH private keys
- AWS credentials
- Service account secrets

For example:

```
DB_HOST=localhost
DB_USER=admin
DB_PASSWORD=secret123
API_KEY=xxxxxxxx
```

Exposed credentials may allow an attacker to access databases, cloud resources, or other connected services.

Operating System Files

Path Traversal may expose sensitive operating system files.

Linux:

```
/etc/passwd
/etc/shadow
```

The `/etc/shadow` file is normally restricted to privileged users.

Windows:

```
C:\Windows\win.ini
C:\Windows\System32\drivers\etc\hosts
```

These files may reveal information about system configuration, users, and network settings.

Arbitrary File Writing

Although Path Traversal commonly results in unauthorized file reading, it may also enable arbitrary file writing when the vulnerable functionality performs file creation or modification.

Depending on the application's permissions and execution environment, this may allow an attacker to:

- Modify application data or configuration files.
- Overwrite existing files.
- Create malicious files or backdoors.
- Alter application behavior.
- Escalate privileges or achieve Remote Code Execution (RCE) in certain circumstances.

The ultimate impact depends on the application's file operations, filesystem permissions, and other security controls.

Reading Arbitrary Files via Path Traversal

Overview

A common use of Path Traversal is to read arbitrary files from a server. This vulnerability occurs when an application uses user-controlled input to construct a file path without adequately validating or restricting the resulting path.

Example Scenario

Consider a shopping application that displays product images. The server returns the following HTML:

```
<img src="/loadImage?filename=218.png">
```

The browser requests the image:

```
GET /loadImage?filename=218.png HTTP/1.1
```

The application receives `218.png` as the value of the `filename` parameter.

How the Application Reads the File

Assume that product images are stored in:

```
/var/www/images/
```

The application appends the supplied filename to this base directory:

```
$file = "/var/www/images/" . $_GET['filename'];
readfile($file);
```

For a normal request, the resulting path is:

```
/var/www/images/218.png
```

The application reads the file and returns its contents to the browser.

Exploiting the Vulnerability

The vulnerability arises because the application trusts the supplied filename without ensuring that the resulting path remains inside the intended directory.

An attacker can replace the image filename with a traversal payload:

```
../../../etc/passwd
```

The resulting request is:

```
GET /loadImage?filename=../../../etc/passwd HTTP/1.1
```

The application constructs the following path:

```
/var/www/images/../../../etc/passwd
```

Understanding Path Traversal Sequences

On Unix-like systems, the sequence `../` refers to the parent directory. Each occurrence moves one directory level upward during path resolution.

Starting from:

```
/var/www/images/
```

The traversal proceeds as follows:

```
/var/www/images/../  -> /var/www/
/var/www/../         -> /var/
/var/../             -> /
```

The remaining path identifies the target file, `etc/passwd`. Therefore, the combined path resolves to:

```
/etc/passwd
```

This process is known as **path resolution**. Path normalization simplifies path components such as `.` and `..`; canonicalization may additionally resolve filesystem-specific details, such as symbolic links, depending on the API used.

Reading Arbitrary Files

If traversal succeeds, the application may expose any file that its effective operating system permissions allow it to access. The impact depends on the application's privileges and the files available to the process.

Example Target: `/etc/passwd`

On Linux and other Unix-like systems, `/etc/passwd` is a standard file containing account information, including:

- Username
- User ID (UID)
- Group ID (GID)
- Home directory
- Login shell

It is commonly used to demonstrate file-reading vulnerabilities because it is generally readable by unprivileged users. Password hashes are normally stored separately in `/etc/shadow`, which typically requires elevated privileges.

Windows Path Traversal

Windows paths commonly use backslashes (`..\`) for parent-directory traversal. For example, a potential target is:

```
C:\Windows\win.ini
```

A traversal payload may use backslashes:

```
..\..\..\Windows\win.ini
```

Some applications and Windows APIs also accept forward slashes:

```
../../../Windows/win.ini
```

Support for either separator depends on how the application, framework, and filesystem API process the supplied path.

Path Traversal: Common Obstacles and Bypass Techniques

Overview

Applications often implement defenses against Path Traversal by removing or blocking traversal sequences, restricting paths to a specific directory, or enforcing file extensions. However, these defenses may be bypassed when validation is incomplete or different components interpret the input differently.

Request Processing and Filtering

A request may pass through several layers before the application accesses the filesystem:

```
Browser
   |
   v
Web Server / Proxy
   |
   v
Framework
   |
   v
Application Code
   |
   v
Filesystem
```

These layers may perform URL decoding, input validation, path normalization, and file access. Differences in their processing order can introduce security weaknesses.

Absolute Path Bypass

Problem

An application may block traversal sequences such as `../` without preventing absolute filesystem paths.

For example:

```
$filename = str_replace("../", "", $filename);
```

Bypass

Instead of using traversal sequences:

```
../../../etc/passwd
```

an attacker may supply the absolute path directly:

```
/etc/passwd
```

Why It Works

An absolute path identifies a location from the filesystem root and does not require `../`. If the application accepts absolute paths and does not enforce a permitted base directory.

Nested Traversal Sequences

Problem

Some applications remove traversal sequences rather than rejecting invalid input. This can be unsafe if the resulting string is not validated again.

Bypass

A nested sequence such as:

```
....//
```

contains an inner `../` sequence. If that inner sequence is removed, the remaining characters can form another traversal sequence.

For example, a single non-recursive replacement of `../` in:

```
....//....//etc/passwd
```

can produce:

```
../../etc/passwd
```

Why It Works

The filter transforms the input into a valid traversal sequence instead of eliminating the underlying risk. The technique depends on the exact filtering logic and whether the transformed result is validated.

URL Encoding and Double URL Encoding

URL Encoding

In some contexts, a web server or another intermediary may filter traversal sequences before passing the input to the application.

The characters in `../` can be URL-encoded as:

```
../       -> %2e%2e%2f
```

Here, `%2e` represents a period and `%2f` represents a forward slash.

If an upstream component checks only for literal traversal sequences before decoding, the encoded value may bypass that check. A later decoding step may restore `../`.

Double URL Encoding

Double encoding encodes the percent signs in an already encoded value:

```
../              -> %2e%2e%2f
%2e%2e%2f        -> %252e%252e%252f
```

A possible processing sequence is:

```
Client:
%252e%252e%252f

First decoding:
%2e%2e%2f

Second decoding:
../
```

This bypass may work when a filter checks the value after one decoding pass, while another component decodes it again later. Success depends on the actual request-processing pipeline.

Non-standard Encodings

Alternative encodings have historically bypassed some implementations that interpreted encoded separators inconsistently.

Examples include:

```
..%c0%af
..%ef%bc%8f
```

These sequences have been accepted as alternative representations of a slash by some legacy or non-standard implementations. They are not reliable, universally valid encodings and may be rejected by modern software.

Burp Suite

Burp Suite Professional includes a predefined Burp Intruder payload list named:

```
Fuzzing - path traversal
```

It contains traversal-related payloads, including encoded variants

Validation of the Start of a Path

Problem

An application may require the supplied path to start with a specific base directory, such as:

```
/var/www/images
```

A weak implementation might check only the string prefix:

```
str_starts_with($filename, "/var/www/images")
```

Bypass

An attacker may provide:

```
/var/www/images/../../../etc/passwd
```

Why It Works

The raw input begins with the expected directory, so a prefix-only check may accept it. However, path resolution can produce:

```
/etc/passwd
```

The application validated the string's beginning rather than verifying the final resolved path.

A secure check must ensure that the canonical path remains inside the intended directory.

File Extension Validation and Null Byte Injection

Problem

An application may require a particular extension, such as `.png`, or append it automatically:

```
$file = $_GET['filename'] . ".png";
```

For example, the input:

```
../../../etc/passwd
```

becomes:

```
../../../etc/passwd.png
```

which ordinarily refers to a different file.

Historical Null Byte Bypass

A null byte has the value `0x00` and can be represented in a URL as `%00`.

Historically, some applications and C-based file APIs treated the null byte as the end of a string. An attacker could attempt to supply:

```
../../../etc/passwd%00.png
```

If one component validated the extension but a later component interpreted the null byte as a string terminator, the effective filename could become:

```
../../../etc/passwd
```

The required extension would then be ignored by that legacy component.

Modern Considerations

Modern runtimes and filesystem APIs generally reject null bytes in file paths rather than treating them as terminators. For example, modern PHP file operations reject paths containing null bytes.

Consequently, this is primarily a historical technique relevant to legacy software and controlled training environments.

Summary of Bypass Techniques

Defense	**Weakness**	**Potential Bypass**
Block `../`	Absolute paths remain accepted	`/etc/passwd`
Strip traversal sequences	Removal may create a new traversal sequence	Nested traversal, such as `....//`
Filter encoded input	Components may decode at different stages	URL or double URL encoding
Check the path prefix `../../../etc/passwd`	The resolved path may escape the directory	`/var/www/images/`
Enforce a file extension	Legacy string handling may terminate at a null byte	`%00` (historical)

Path Traversal Prevention

Avoid User-Controlled File Paths

The strongest defense is to avoid passing user-controlled paths to filesystem APIs. Prefer indirect identifiers that the application maps to predefined files.

Instead of:

```
GET /image?filename=cat.png
```

use:

```
GET /image?id=5
```

For example:

```
$id = $_GET['id'];

$files = [
    1 => "/var/www/images/logo.png",
    2 => "/var/www/images/cat.png"
];

if (isset($files[$id])) {
    readfile($files[$id]);
}
```

The user selects an allowed identifier but never supplies the filesystem path.

Validate Input and Use a Trusted Base Directory

If user input must influence a path, validate it against a strict allowlist of permitted identifiers, filenames, or characters. Reject unexpected input rather than trying to sanitize it by removing strings such as `../`; blacklist filtering can be bypassed by alternative encodings or nested traversal sequences. Construct the path relative to a fixed, trusted base directory. However, prepending a base directory alone is insufficient: an input such as `../../../etc/passwd` may still escape it after path resolution.

Canonicalize and Verify Directory Containment

Canonicalization resolves path components such as `.` and `..`; depending on the API, it may also resolve symbolic links. For example:

```
/var/www/images/../../../etc/passwd
```

may resolve to:

```
/etc/passwd
```

After canonicalization, verify that the resolved path remains inside the authorized base directory. Do not rely on a naive prefix check: a path such as `/var/www/images-backup/file.txt` begins with the string `/var/www/images` but is not inside that directory.

Java Example

```
File base = new File(BASE_DIRECTORY).getCanonicalFile();
File file = new File(base, userInput).getCanonicalFile();

String basePath = base.getPath();
String filePath = file.getPath();
String allowedPrefix = basePath + File.separator;

if (filePath.startsWith(allowedPrefix)) {
    // Process the file
} else {
    // Reject the request
}
```

The code canonicalizes both paths and requires the resulting file path to begin with the base directory followed by a path separator. This prevents a simple string-prefix collision and rejects paths that resolve outside the intended directory.

This example is illustrative rather than a complete defense for every environment. Canonical-path checks can be affected by symbolic links and race conditions between validation and file access. Production code should use suitable path-aware APIs, control filesystem permissions, and avoid allowing untrusted users to modify the directory structure between checking and opening a file.

Best Practices

- Avoid user-controlled filesystem paths whenever possible.
- Prefer indirect identifiers and strict allowlists.
- Reject invalid input instead of modifying it with simple string replacement.
- Build paths relative to a trusted base directory.
- Canonicalize the path before validating its location.
- Verify that the resolved path remains inside the authorized directory.
- Apply appropriate filesystem permissions and account for symbolic links and race conditions.